

MIPS CPU on FPGA - Final Report

Group 5 - 汤铁峰 1820232065, 邝振锋 1820232069, 蔡立根 1820232081

2026-09-17

计算机组成原理课程设计

1. Preparation	2
2. Final Implementation	8

1. Preparation

1.1 Preparation

- MIPS canonical 5-stage pipeline

1.1 Preparation

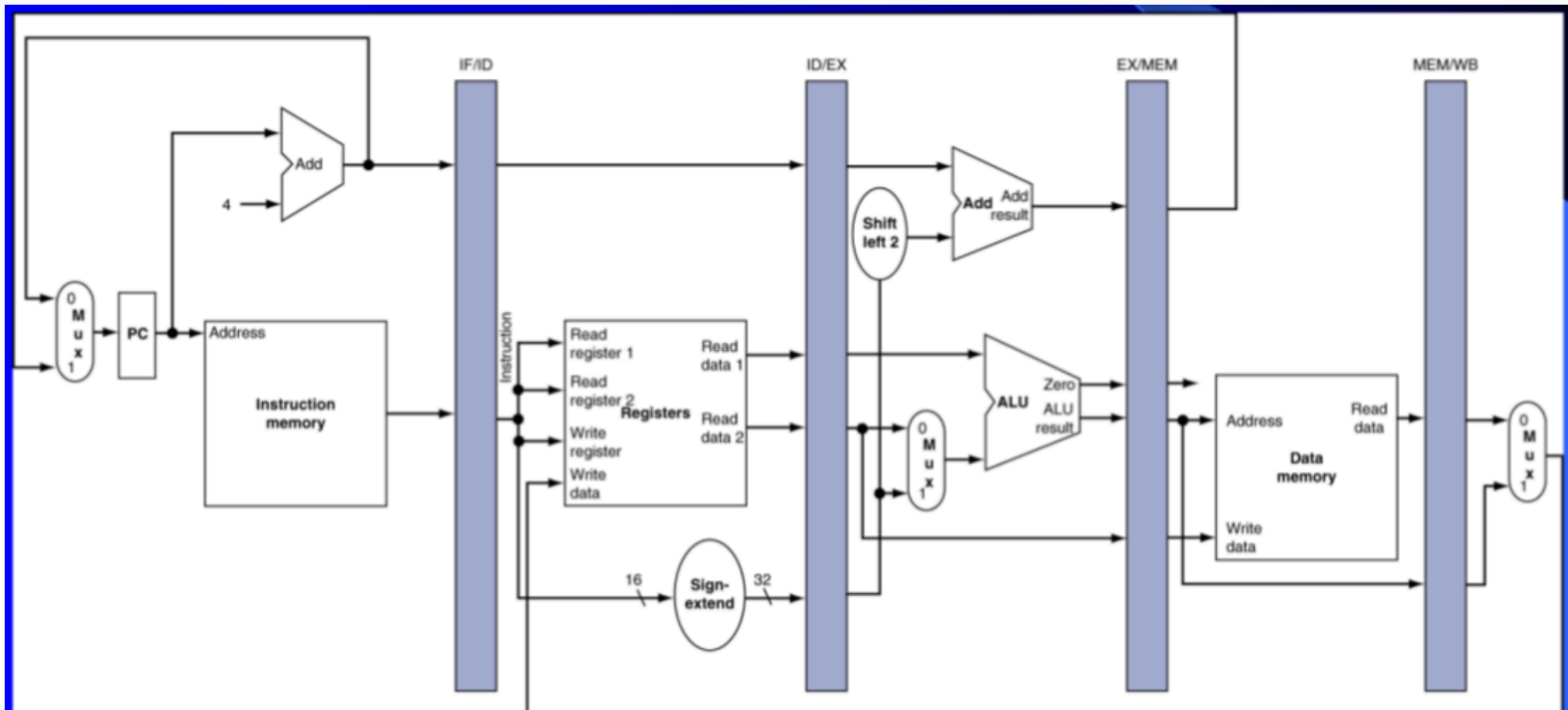


Figure 1: Pipeline Schematic (from PPT)

1.1 Preparation

- Pipeline hazards: structural, data, and control

1.1 Preparation

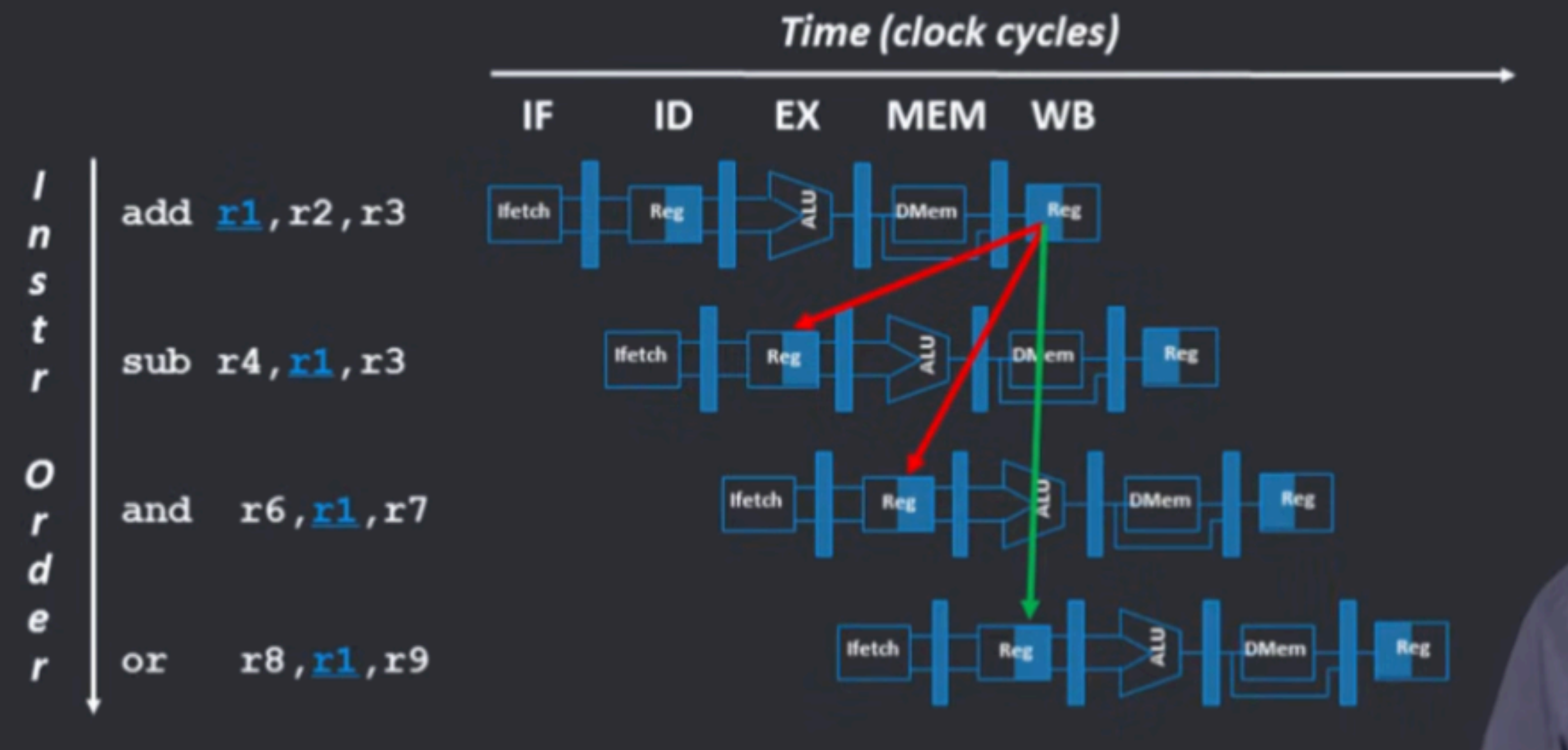


Figure 2: Data Dependency: Data Hazard Example (<https://www.youtube.com/@prof.dr.benh.juurlink5459>)

1.1 Preparation

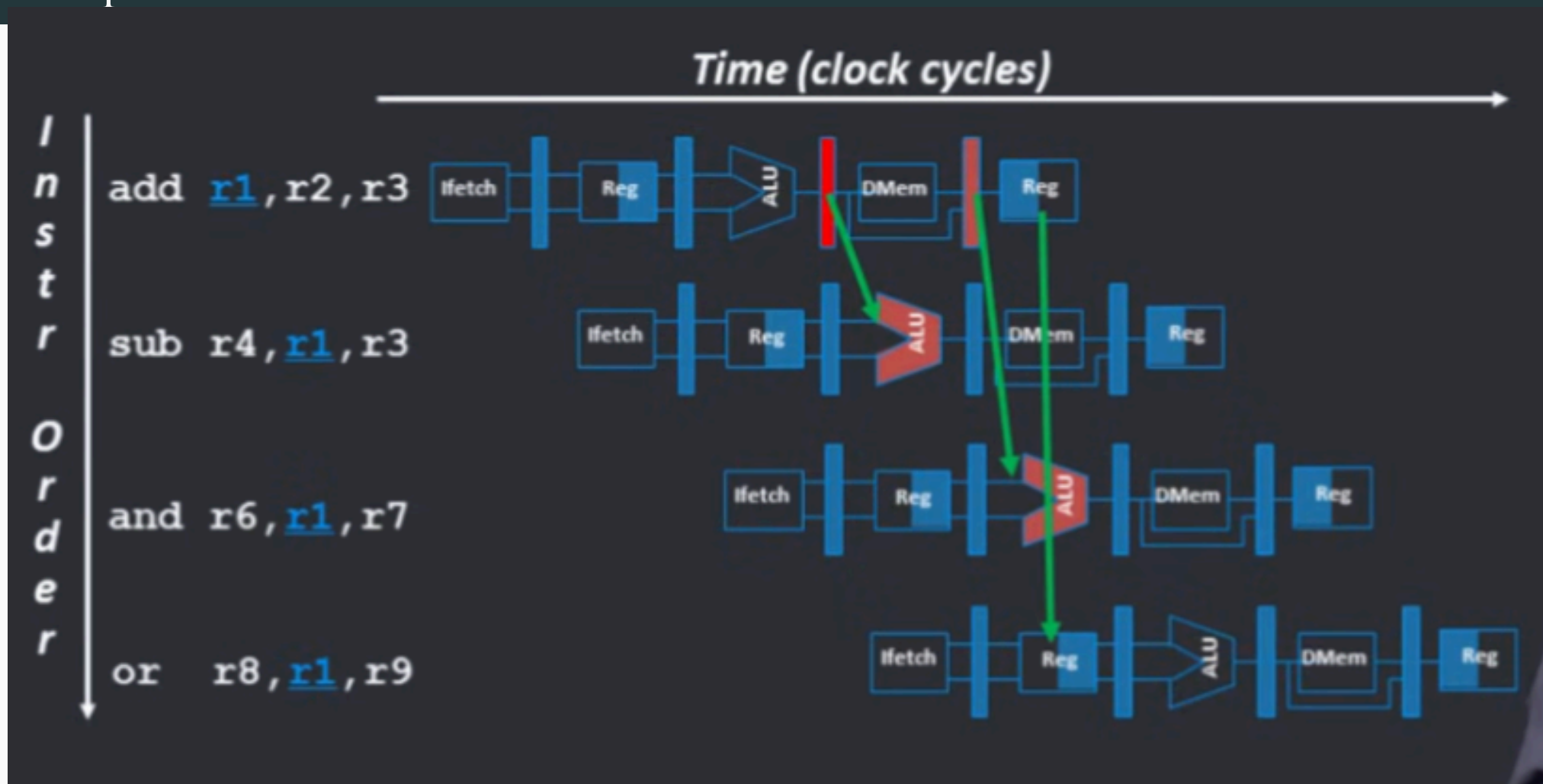


Figure 3: Forwarding: Data Hazard Solution (<https://www.youtube.com/@prof.dr.benh.juurlink5459>)

2. Final Implementation

2.1 Overview of our design

- A **larger** MIPS subset, now enough for clang-generated code: R-type (addu subu slt sltu and or xor nor sll srl sra jr), I-type (addiu lui lw sw beq bne slti andi ori blez bgez), J-type (j jal). In total, **25** instructions.
- Five stages — **IF, ID, EX, MEM, WB** — with a new instruction starting every cycle, so the CPI is close to 1.
- Pipeline registers IF/ID, ID/EX, EX/MEM, MEM/WB carry the data **and the control bits** forward to the later stages.
- Separate instruction and data SRAM (Harvard-style) — no structural hazards.
- Branches are resolved in ID and the following instruction is flushed, so there is no delay slot; jal links PC + 4.

The datapath is the single-cycle one, cut into stages:

```
IF -> ID -> EX -> MEM -> WB
PC -> imem -> control/regfile -> alu -> (dmem) -> write-back
```

2.2 Pipeline registers and data flow

- IF/ID carries PC and the fetched instruction into decode.
- ID/EX carries the register values, immediates, destination and control signals; the ALU op is pre-decoded here.
- EX/MEM carries the ALU result and the store data; the load data is captured at this boundary because the SRAM read is combinational.
- MEM/WB carries the final result into the register-file write-back.
- Valid bits and allowin signals let a stage stall (load-use) or flush (branch) without disturbing the stages behind it.

2.2 Pipeline registers and data flow

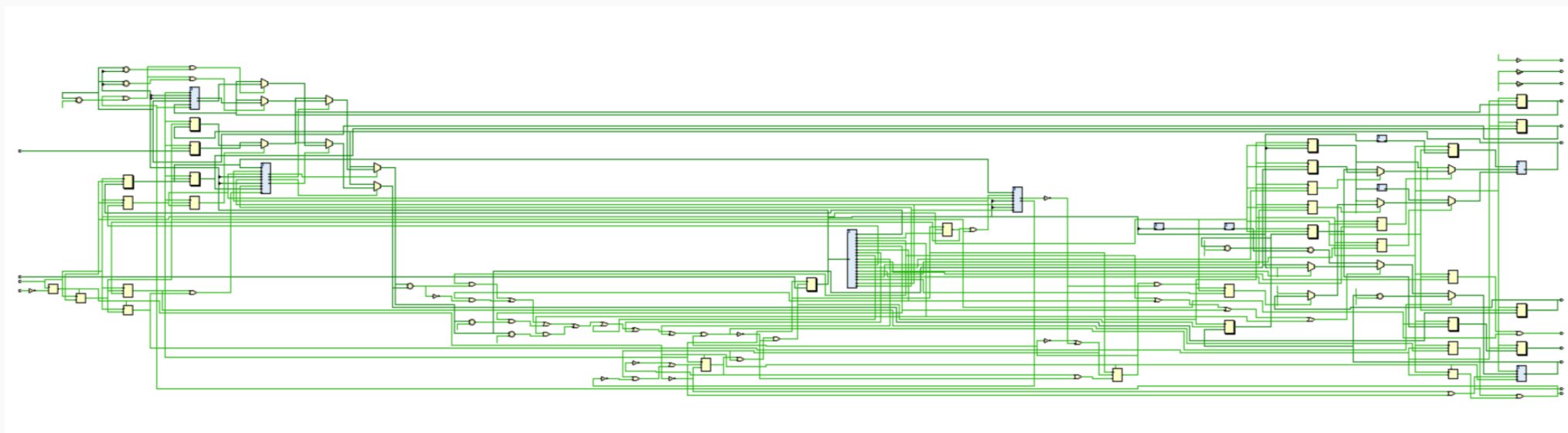


Figure 4: Vivado RTL analysis

2.3 Module breakdown

module	responsibility	kind
pc	Program counter (IF), reset + enable	sequential
imem	Instruction SRAM, word-indexed (Harvard)	comb. read
control	decodes the ID instruction → control signals + branch type	comb.
alu_control	decoded instruction → 12-bit one-hot ALU operation	comb.
regfile	32 × 32 regs, 2 read / 1 write port, \$zero masked	sequential
sign_extend	16 → 32-bit sign extension of immediates	comb.
shift_left2	<<2 used for branch offsets and jump targets	comb.
alu	add / sub / and / or / xor / nor / slt / sltu / shifts / lui	comb.
forward	EX/MEM → ID forwarding selects	comb.
hazard	load-use stall: freeze IF and ID for one cycle	comb.
dmem	Data SRAM, sync write / comb. read	sequential
mips_pipeline	top-level: wires the stages and pipeline registers	structural

2.4 Hazard handling

- **Data hazards:** forward selects the freshest operand from the EX result or from MEM/WB, so most dependencies cost no cycles.
- **Load-use hazard:** a load result is not ready in EX, so hazard stalls IF and ID for one cycle and the value is then forwarded from MEM/WB.
- **Control hazards:** beq/bne/blez/bgez/j/jal/jr are resolved in ID; the instruction fetched behind them is flushed (no delay slot).

2.4 Hazard handling

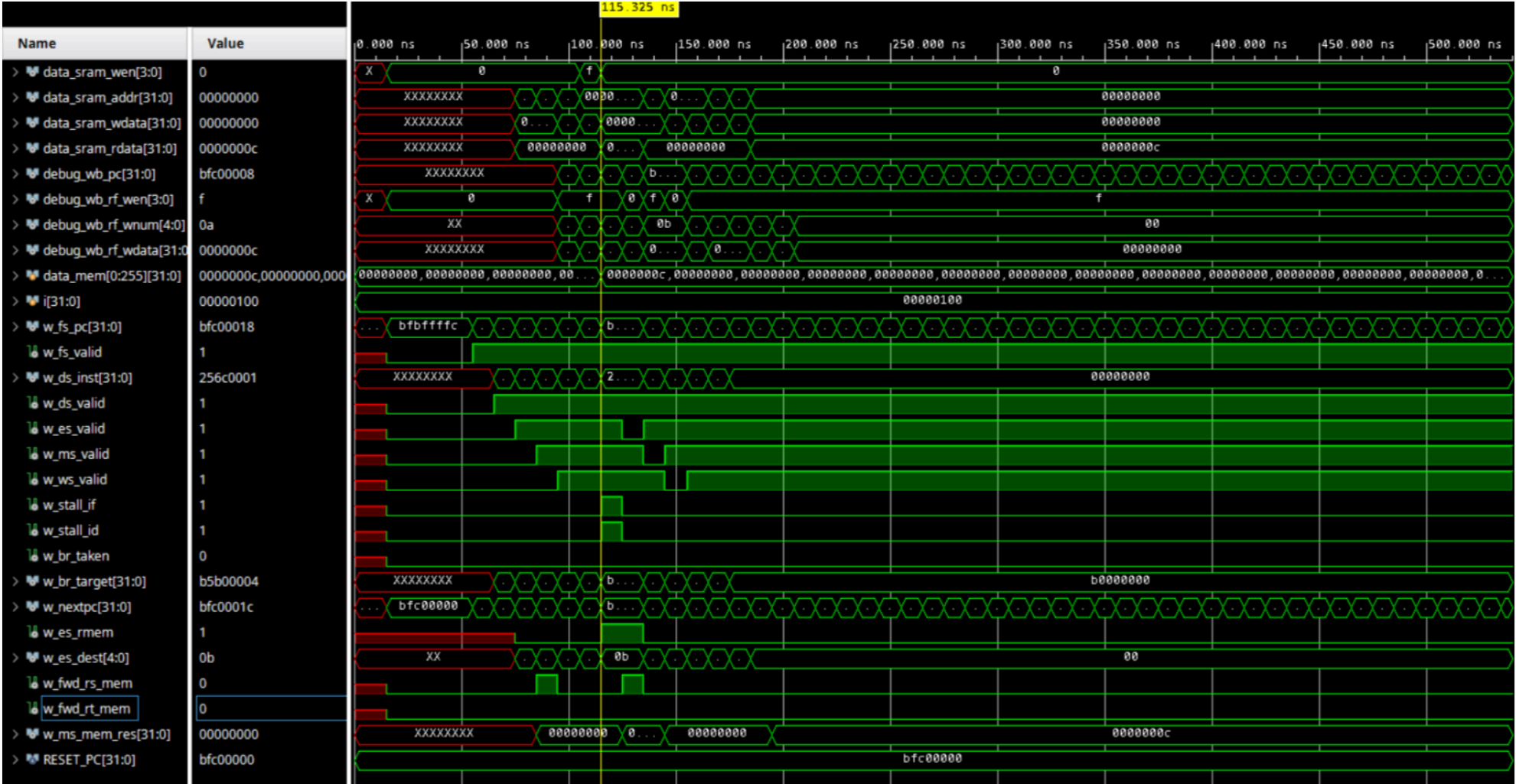


Figure 5: Waveform of a load-use stall

2.4 Hazard handling

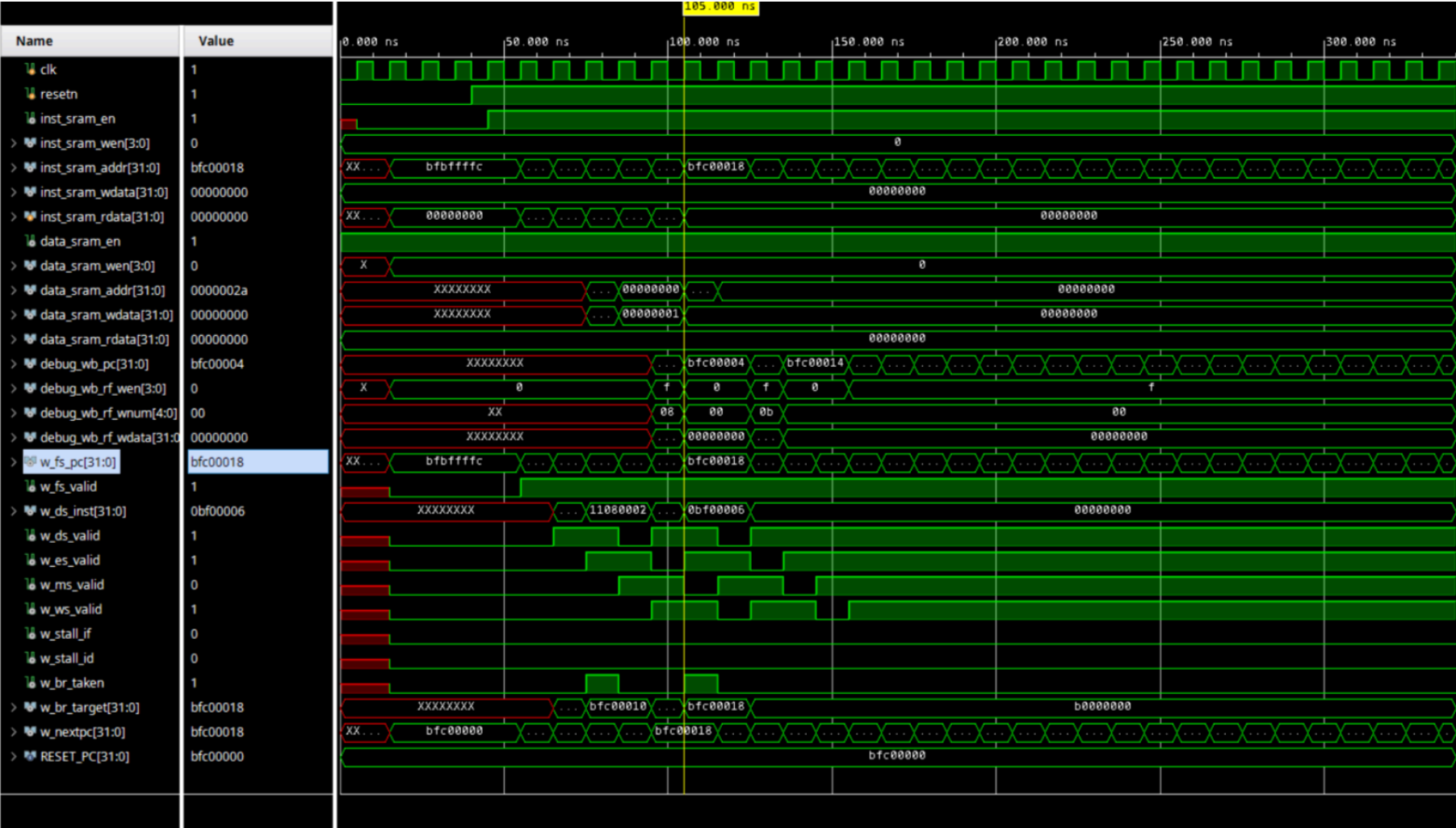


Figure 6: Waveform of when a branch is taken

2.5 Verification: the test programs

The core is exercised by directed testbenches and a compiler-generated program:

tb_pipeline_cpu	hazard + forwarding (addu / sw / lw / subu / and / or / xor)
tb_branch_cpu	branch flush and target (beq)
tb_jal_cpu	subroutine link / return (jal + jr, links PC + 4)
tb_isa_ext	slti / andi / ori / j / blez / bgez + load capture
tb_bubble_sort	bubble sorts an 8-word array in data memory
tb_fibonacci	stores fib(0..9) iteratively

Program was written in MIPS asm and then compiled to machine code with MARS assembler.

Performance counters are exposed for every run.

```
# run 10000
branch test: t0=1 t1=0 t2=0 t3=42
performance: cycles=29 retired=22 load_use_stalls=0 taken_branches=2
PASS: branch flush/target test
```

Figure 7: Example result: tb_branch_cpu.v

2.5 Verification: the test programs

```
unsorted array: 5 3 9 1 7 2 8 4
tb_bubble_sort: running (waiting for completion flag)...
sorted array: 1 2 3 4 5 7 8 9
PASS: bubble sort
performance: cycles=528 retired=436 load_use_stalls=28 taken_branches=60
```

Figure 8: Example result: tb_bubble_sort.v

```
tb_fibonacci: running (waiting for completion flag)...
fib{0..9}: 0 1 1 2 3 5 8 13 21 34
PASS: fibonacci
performance: cycles=134 retired=118 load_use_stalls=0 taken_branches=12
```

Figure 9: Example result: tb_fibonacci.v

2.6 Performance

The core exposes four simulation counters; CPI is `cycle_count / retired_count`:

- `cycle_count` — active cycles after reset
- `retired_count` — instructions that reach write-back
- `load_use_stall_count` — cycles lost to a load-use stall
- `taken_branch_count` — taken branches/jumps (each flushes one fetch)

test	cycles	retired	stalls	CPI
hazard / forwarding	49	43	1	1.14
branch flush / target	29	22	0	1.32
bubble sort (8 words)	528	436	28	1.21
fibonacci (0..9)	134	118	0	1.14

- Straight-line code runs at $\text{CPI} \approx 1.14$; fibonacci is the same ($134 / 118$).
- Bubble sort costs 1.21: 28 load-use stalls and 60 taken-branch flushes over 436 instructions.
- The branch test isolates the flush cost — two taken branches give 1.32.
- `jal / isa_ext` end in an infinite loop, so their cycle count keeps growing after the program finishes — their CPI is not a clean efficiency figure.

2.6 Performance

- On a real workload the pong program runs 41k cycles per 60 Hz frame.

2.7 Bonus: a playable demo

- Verilator + raylib frontend: `pong.c` is compiled for MIPS-I with clang's integrated backend and loaded at the reset vector.
- Memory map: `0x00000000` RAM · `0x00010000` 64×32 framebuffer · `0x00020000` input · `0x00020004` cycle counter · `0x00020008` frame-done.
- Runs one game frame per rendered 60 Hz frame (41k cycles/frame); verified headless over 250 frames (bounces, paddle clamp, AI tracking, serves).

2.7 Bonus: a playable demo



Figure 10: Our CPU running pong through Verilator

Thank You